

How I made a fast game agent make more careful choices

Interview brief — September 25, 2026

Explain it like I am five

Imagine a kid playing a card game very quickly. At each turn, the kid can point to a card, but speed does not mean the choice is good. I gave the kid a short checklist: “Will this keep me alive? Will it help me win? Am I wasting something valuable?” The checklist is answered in one trip, so the game stays fast. I also made the kid look at the *kind* of choice in front of them: picking a route is different from choosing a card or buying something in a shop.

The computer still makes a judgment. My code supplies the actual game facts, checks the available actions, combines the judgments consistently, and prevents one clearly fatal end-turn choice when the game math shows a surviving alternative.

What I would say in 30 seconds

“I worked on a Slay the Spire 2 agent that uses Jev, a fast decision model. The original project already connected the game to Jev; I built and evaluated a decision layer on top. Instead of asking several sequential questions or using one vague ‘best move’ question, I put the legal moves and focused safety, progress, and waste questions into one model request. I then combined the answers in code. I also added screen-specific context for routes, rewards, shops, and combat, plus a narrow guard against ending a turn when calculated damage would kill the player. On a small recorded-state benchmark, the earlier one-call policy scored 90/100 at 325 ms median versus the upstream multi-call policy’s 24/30 at 758 ms median. Those scores come from small, different sample counts, so I present the speed result as promising, not a win-rate claim. The newest contextual policy matched the existing fast policy on the checks we ran; it has not yet proven better full-game play.”

Why I did it

The agent already played quickly, but some choices were strategically weak. A fast model can rank options without knowing that a setup card has no payoff, an energy card is useless with an empty hand, or a route risks an elite the deck cannot handle. Sending the model more prose was not enough. I wanted the decision process to be **specific to the screen, measurable, and safe to run** without losing its speed.

Training a new Jev model was not the practical first move: we did not have a large, labeled collection of successful and failed game decisions. A game mod was already in the stack. The existing STS2MCP mod exposes game state and accepts actions; it is a bridge, not the strategy. The improvement opportunity was the policy between that bridge and Jev.

How it works, step by step

1. **Read the game.** The existing bridge reports the visible board, hand, enemies, map, and other current facts. The planner turns that state into legal candidate actions. It never asks Jev to invent a move.
2. **Ask Jev focused questions in one request.** For combat, the request includes a broad move choice plus a safety, progress, and wasted-cost judgment for each candidate. Jev returns typed choices and probability-like scores. It is a decision model, not a text-writing chatbot.
3. **Combine the answers in code.** The one-call policy normalizes the factors, ignores differences too small to trust, and gives wasted-cost risk extra weight. This caught cases such as spending health or energy for a payoff that could not be collected. If a factor is missing, the newest policy falls back to Jev's broad legal move choice instead of treating missing evidence as a free pass.
4. **Use the right checklist for the screen.** A card reward is compared with skipping and with the actual deck; a shop purchase is compared with the budget and other purchases; a map route is judged using visible connections and recent damage; combat uses enemy intents and recent observations. Recent memory is capped so the request stays bounded. Unknown rooms, draws, and future rewards remain unknown.
5. **Apply one deterministic safety rule.** If Jev picks "end turn," and a calculated forecast says that would kill the player while another calculated continuation survives, the code selects the surviving continuation. It does not overrule uncertain forecasts or claim that every other move is optimal.
6. **Execute once, then observe again.** Before sending the chosen command, the server checks that the game state has not changed. It never automatically retries a game command after an uncertain timeout, which avoids playing a card twice. Duplicate *decision* requests can be safe because they do not change the game.

What I implemented versus what I inherited

I built the one-call factored decision layer, scoring experiments, latency handling, and the newer contextual policy in this fork. I added recorded-state comparisons, tests, and the preview that explains the latest result. The original project by **Alex Meckes** provided the Slay the Spire 2 integration, planner, fixtures, and grading suite. **Yikun Ji's STS2MCP** provides the game mod/bridge. I would credit both explicitly in an interview.

Results I can defend

- The earlier factored policy used **one model call per decision**, versus **2.40 calls on average** for the upstream policy in its measured sample. Median decision time was **325 ms versus 758 ms**. Its graded result was **90/100 versus 24/30**. The sample sizes differ, and the quality check covers only selected first actions, so this is a development signal rather than proof of a better win rate. [Source: sweep findings](#).
- On a separate 30-decision comparison with identical recorded states, Jev scored **27/30** while several larger models scored **29/30**. This small gap is within the measured noise

floor; the defensible finding is that Jev was much faster and cheaper *on that test*, not smarter than those models. [Source: measurements](#).

- For the **newest contextual policy**, the final paired replay passed **15/18** known-mistake checks for **both** policies. Both failed the same Beckon preferred-action check. A separate set of **four immediate-survival states** passed **4/4 for both**. The comparison used one model call per decision and did not send commands to the game. These are small, selected cases; there is **no measured full-run win-rate improvement yet**. [Source: latest local replay](#) and [fresh replay](#).
- The local automated suite passes **386 tests**, measured on 2026-09-29 with npm test. The count grows as cases are added, so re-run the suite rather than trusting the number recorded here. That verifies code behavior; it does not prove stronger gameplay. [Source: policy tests](#).

If they ask, “Did you train the model?”

“No. I improved the *decision system around the model*: what facts it sees, which small questions it answers, how those answers are combined, and when deterministic game math should overrule a clearly fatal choice. Model training would need a much larger, trustworthy dataset and held-out evaluation. The current results do not justify claiming training would beat this approach.”

If they ask, “Did you write a game mod?”

“No. I used the existing STS2MCP mod as the bridge to read game state and issue actions. My contribution is the policy and evaluation layer on top of it. Keeping those layers separate let me test decisions on recorded states without moving the live game.”

If they ask, “What was the hardest lesson?”

“A model score can look precise while being unhelpful. When all candidates had almost the same factor score, normalization exaggerated a tiny difference into a strong vote. I added a deadband so nearly equal scores are neutral. I also learned that a missing factor must trigger a fallback, and that a safety override should require calculated evidence rather than a guess.”

If they ask, “What would you do next?”

“I would measure full runs on matched seeds across many runs, separating survival, floors reached, bosses defeated, and win rate from latency and cost. I would hold out new runs when tuning the policy, inspect failure cases, and only then decide whether a labeled-data or training project is justified. Until then, I call the newest change a tested policy improvement in structure and safeguards, not a proven gameplay improvement.”

One sentence to remember

“I made Jev’s fast decisions more structured and safer, then measured the result honestly: the earlier policy got faster on recorded states, while the newest policy still needs full-run evidence before I can say it plays better.”